

Politechnika Warszawska

W Y D Z I A Ł E L E K T R Y C Z N Y



Instytut Elektrotechniki Teoretycznej i Systemów Informacyjno-Pomiarowych

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana
w specjalności Inżynieria Oprogramowania

Analiza metod mitygacji podatności dużych modeli językowych

Filip Sosnowski

numer albumu 319105

promotor

dr inż. Robert Szmurło

WARSZAWA 2026

Analiza metod mitygacji podatności dużych modeli językowych

Streszczenie

Słowa kluczowe:

Analysis of methods for mitigating vulnerabilities in Large Language Models

Abstract

Keywords:

Spis treści

1	Wstęp	9
1.1	Motywacja	9
1.2	Cel i zakres pracy	9
1.3	Zawartość pracy	10
2	Ataki na duże modele językowe	11
2.1	Zagrożenia bezpieczeństwa	11
2.2	Taksonomia ataków promptowych	12
2.3	Metody bezpośrednich ataków promptowych	12
2.3.1	Ataki oparte na optymalizacji	12
2.3.2	Metody manualne	13
2.3.3	Automatyczny red teaming oparty na modelach	13
3	Metody obrony przed atakami na duże modele językowe	15
3.1	Taksonomia metod obrony	15
3.2	Metody na poziomie promptu	15
3.3	Metody na poziomie modelu	15
3.4	Metody na poziomie wyjścia	15
4	Projekt i implementacja środowiska badawczego	17
4.1	Cel i założenia eksperymentu	17
4.2	Architektura systemu testowego	17
4.3	Wybrane modele językowe	17
4.4	Zbiory danych	17
4.5	Metryki oceny	17
4.6	Implementacja wybranych metod obrony	18
5	Analiza wyników badań	19
5.1	Skuteczność ataków bez zastosowania metod mitygacji	19
5.2	Skuteczność poszczególnych metod obrony	19
5.3	Analiza kompromisu między bezpieczeństwem a użytecznością modelu	19
5.4	Analiza opóźnień wprowadzanych przez obronę	19

5.5	Dyskusja wyników i ograniczenia badania	19
6	Podsumowanie	21
6.1	Wnioski	21
6.2	Możliwości dalszych badań	21
	Bibliografia	23
	Wykaz skrótów	25
	Spis rysunków	27
	Spis tabel	29

Rozdział 1

Wstęp

1.1 Motywacja

Motywacją do podjęcia tego tematu jest między innymi globalny wzrost zainteresowania technologiami wykorzystującymi sztuczną inteligencję (ang. Artificial Intelligence (AI)). Według raportu przygotowanego przez Uniwersytet Stanforda, generatywna sztuczna inteligencja (ang. Generative Artificial Intelligence (GenAI)) została zaadaptowana przez 53% populacji w ciągu trzech lat, co czyni ją jedną z najszybciej wdrażanych technologii w historii [8]. Ważną częścią tych systemów stają się duże modele językowe (ang. Large Language Model (LLM)), które używane są do tworzenia zaawansowanych wyszukiwarek, chatbotów czy narzędzi wspomagających wytwarzanie oprogramowania [10]. Stosowanie modeli językowych na skalę masową sprawia, że zagwarantowanie ich bezpieczeństwa ma wyjątkową wagę. Spośród zidentyfikowanych zagrożeń dla systemów opartych na LLM, szczególne znaczenie mają ataki typu wstrzykiwanie poleceń (ang. prompt injection). Polegają na celowym manipulowaniu zachowaniem lub odpowiedzią modelu językowego w sposób niezamierzony poprzez polecenie użytkownika [6]. Istotność tego ryzyka potwierdza klasyfikacja przygotowana przez organizację Open Worldwide Application Security Project (OWASP), która w swoim zestawieniu uznaje tego typu ataki za największe zagrożenie dla bezpieczeństwa we współczesnych aplikacjach wykorzystujących modele językowe [6]. Z tego względu główną motywacją do podjęcia niniejszych badań jest chęć analizy i porównania skuteczności metod mitygacji tej krytycznej podatności w obliczu szybkiego tempa integracji systemów GenAI i potrzeby zapewnienia wysokiego poziomu bezpieczeństwa dużych modeli językowych.

1.2 Cel i zakres pracy

Głównym celem pracy jest przeprowadzenie analizy porównawczej i ocena skuteczności wybranych metod mitygacji podatności dużych modeli językowych na ataki typu prompt injection. Realizacja wymaga implementacji zróżnicowanych podejść do obrony, takich jak walidacja danych wejściowych, zewnętrzne moderacyjne modele językowe czy modyfikacji stanów wewnętrznych sieci metodą

inżynierii aktywacji (ang. activation engineering). W celu sprawdzenia bezpieczeństwa modeli należy również poddać je różnym wektorom ataku. Ważnym krokiem jest również opracowanie procedury testowej opartej na zbiorze danych zawierającym zapytania niedozwolone oraz bezpieczne. Rezultatem pracy będzie wskazanie optymalnych mechanizmów obronnych dla wybranych modeli językowych przy zachowaniu zadowalającej jakości ich działania.

1.3 Zawartość pracy

Praca składa się z x rozdziałów. We wstępie omówiono motywację do podjęcia tematu, określono cele do zrealizowania oraz opisano strukturę pracy. Następny rozdział zawiera W rozdziale trzecim omówiono W podrozdziałach czwartej części ...

Rozdział 2

Ataki na duże modele językowe

2.1 Zagrożenia bezpieczeństwa

Wraz z rozwojem dużych modeli językowych pojawiają się ich nowe, groźne podatności. Udoskonalaniu mechanizmów bezpieczeństwa towarzyszy również rozwój coraz bardziej zaawansowanych metod ataku. Stanowi to istotne wyzwanie dla twórców systemów AI, którzy dążą do rozbudowy funkcjonalności modeli jednocześnie minimalizując ryzyko ich nadużycia. Potężne możliwości generowania tekstu przez duże modele językowe niosą ze sobą również zagrożenie takie jak tworzenie szkodliwych treści, zarówno w sposób nieintencjonalny jak i w wyniku przeprowadzonego ataku [14]. Generowane w ten sposób niedozwolone odpowiedzi można uporządkować w cztery kategorie [14]. Jedną z nich może być toksyczność obejmująca wulgaryzmy, wyzwiska czy uprzedzenia kulturowe. Drugą kategorią jest nękanie, którego celem jest zastraszanie czy kierowanie gróźb. Kolejną kategorią jest mowa nienawiści. Ostatnią kategorię tworzą treści obarczone uprzedzeniami, które skierowane są do osób o określonej rasie, płci, religii czy orientacji. Rozwój bezpieczeństwa dużych modeli językowych jest kluczowy, by niwelować możliwość generowania takich treści i dostarczania ich użytkownikom.

Według organizacji OWASP najistotniejszym zagrożeniem dla takich systemów są ataki typu wstrzykiwanie poleceń [6]. Podatność ta występuje, gdy dane wejściowe przekazane przez użytkownika modyfikują zachowanie lub odpowiedź modelu w sposób nieprzewidziany przez twórców. Zagrożenia te można podzielić na dwie podgrupy. Pierwszą z nich są pośrednie ataki typu prompt injection, w których system przyjmuje dane wejściowe ze źródeł zewnętrznych, przykładowo z plików czy stron internetowych zawierających treść, która może wpłynąć na zachowanie modelu w sposób nieoczekiwany [6]. Tego typu ataki mogą prowadzić do zakłóceń prawidłowego działania modelu, blokowania funkcjonalności, skłonienia systemu do odbiegania od pożądanego zachowania na rzecz celów adversarialnych czy wycieku prywatnych danych przetwarzanych przez LLM [10]. Obok ataków pośrednich, istotną podgrupą tych zagrożeń są również bezpośrednie ataki promptowe (ang. direct prompting attacks), które występują w scenariuszu, w którym atakujący jest głównym użytkownikiem systemu i wchodzi w interakcję z modelem poprzez interfejs zapytań [10]. Zagrożeniem wynikającym z takich ataków jest przede wszystkim obejście mechanizmów obronnych i zasad etycznych

wbudowanych przez twórców modelu, przejęcie kontroli nad wywoływaniem zewnętrznych narzędzi, czy również wydobywanie ukrytych informacji z kontekstu rozmowy, takich jak przykładowo instrukcja systemowa (ang. system prompt) [10]. Podklasą ataków bezpośrednich są ataki typu jailbreak, które koncentrują się na umożliwieniu nieuprawnionego użycia poprzez obejście wbudowanych w modelu mechanizmów odmowy, co ułatwia realizację kolejnych, bardziej złożonych ataków [5]. W dalszej części pracy analizie poddano właśnie kategorię ataków bezpośrednich.

2.2 Taksonomia ataków promptowych

2.3 Metody bezpośrednich ataków promptowych

2.3.1 Ataki oparte na optymalizacji

Pierwszą analizowaną kategorią metod bezpośrednich ataków promptowych są ataki oparte na optymalizacji. Opierają się na sformułowaniu matematycznej funkcji celu i wykorzystaniu metod przeszukiwania do wyznaczenia ciągu wejściowego do modelu wymuszający pożądane zachowanie [10]. W szczególności używane są metody gradientowe. Funkcje celu są formułowane tak, by wymusić na modelu wygenerowanie początku odpowiedzi, który ukierunkowany byłby na zgodę na wykonanie otrzymanego złośliwego polecenia [10]. Specjalną kategorią ataków opartych na gradientach są uniwersalne wyzwalacze adversarialne (ang. universal adversarial triggers). Ich celem jest odnalezienie przedrostków lub przyrostków, które po dodaniu do promptu wejściowego doprowadzają do obejścia zabezpieczeń [10]. Ponadto, znalezione w ten sposób cząstki można wykorzystać w atakach na inne LLM. Czyni to modele otwartoźródłowe dogodnym środowiskiem do projektowania takich ataków.

Przykładem techniki wykorzystywanej przy atakach opartych na optymalizacji jest algorytm Greedy Coordinate Gradient (GCG) [15]. Wykorzystuje on wagi modelu otwartoźródłowego. Rozwiązuje problem optymalizacji w dyskretnej przestrzeni tokenów poprzez połączenie technik opartych na gradientach i metod zachłannego przeszukiwania. Algorytm GCG wykorzystuje gradienty funkcji straty względem wektorowych reprezentacji tokenów, aby wytypować pulę najbardziej obiecujących zamienników. Następnie oceniany jest ich rzeczywisty wpływ za pomocą ewaluacji w przód, za każdym razem wybierając modyfikację maksymalizującą szansę na wygenerowanie przez model odpowiedzi wyrażającej aprobatę. Rezultatem jest wskazanie sufiksu, który może zostać dodany do szkodliwego promptu użytkownika. Autorzy w badaniach na własnym zbiorze wykazali, że zoptymalizowane w ten sposób ciągi tokenów cechują się wysoką skutecznością w przełamaniu zabezpieczeń modeli otwartych. W scenariuszu generowania szkodliwych odpowiedzi na niedozwolone polecenia metoda osiągnęła wskaźnik sukcesu ataku na poziomie 99% dla modelu Vicuna-7B [3] oraz 88% dla modelu Llama-2-7B-Chat [9]. Co więcej, na podstawie badań autorów, wygenerowane w taki sposób sufiksy wykazały wysoki poziom przenośności na inne modele zamknięte, gdzie również dzięki nim udało się złamać zabezpieczenia. Dowodzi to, że podatność na ten typ ataku jest powszechna wśród różnych rodzin dużych modeli językowych.

2.3.2 Metody manualne

Kolejną kategorią są manualne metody przeprowadzenia ataku typu jailbreak [10]. W ich skład wchodzi między innymi metody oparte o konflikt celów, które wykorzystują spór między chęcią modelu do wykonania polecenia użytkownika a koniecznością przestrzegania wbudowanych reguł bezpieczeństwa. Wśród takich podejść można wyróżnić między innymi wstrzykiwanie przedrostka, tłumienie odmowy, wstrzykiwanie stylu oraz odgrywanie ról [11]. Wstrzykiwanie przedrostka oparte jest wymuszeniu na modelu rozpoczęcia odpowiedzi od zgody na wykonanie zadania, co może zwiększyć prawdopodobieństwo na kontynuację odpowiedzi zgodnie ze złośliwą intencją użytkownika. Z kolei podejście mające na celu tłumienie odmowy oparte jest na zakazaniu modelowi stosowania zwrotów wskazujących na brak zgody wykonania zadania. Poprzez obniżenie prawdopodobieństwa wygenerowania formuły odmowy, zwiększona zostaje szansa na uzyskanie odpowiedzi na polecenie atakującego. Wstrzykiwanie stylu natomiast polega na narzuceniu ograniczeń na styl lub składnię wypowiedzi, co może utrudnić modelowi użycie standardowych formuł odmowy. Techniki oparte na odgrywaniu ról bazują na wprowadzeniu fikcyjnego kontekstu ma na celu skłonienie modelu do generowania treści, których odmówiłby w standardowym trybie działania, a przykładem może być podejście typu Do Anything Now (DAN) [4].

Inną grupą metod manualnych są ataki oparte o niedopasowanie generalizacji [10]. Polegają na identyfikacji zapytań, które wykraczają poza rozkład danych użytych podczas treningu bezpieczeństwa modelu, ale znajdują się w obszarze jego ogólnych zdolności generatywnych [11]. Powoduje to ominięcie mechanizmów obronnych ze względu na nietypową formę treści polecenia. W ramach takich technik wyróżnić można strategie bazujące na kodowaniu treści prompta (np. base64), wykorzystaniu szyfru przesuwającego (np. ROT13), rozbijaniu złośliwej instrukcji na mniejsze, pozornie nieszkodliwe fragmenty (ang. payload splitting) czy tłumaczeniu polecenia na mało znane języki (np. zulu) [11–13].

2.3.3 Automatyczny red teaming oparty na modelach

Następną kategorią metod bezpośrednich ataków promptowych jest automatyczny red teaming oparty na modelach [10]. Red teaming polega na antagonistycznym testowaniu systemu w celu zidentyfikowania jego podatności. Technika tego ataku opiera się na wykorzystaniu trzech LLM, w skład których wchodzi model docelowy, atakujący oraz oceniający. W metodzie tej klasyfikator oceniający czy odpowiedź jest szkodliwa można potraktować jako funkcję nagrody do trenowania modelu do generowania szkodliwego polecenia, które może ominąć reguły bezpieczeństwa atakowanego innego modelu [10].

Jedną z takich metod jest algorytm Prompt Automatic Iterative Refinement (PAIR) [2]. Jego celem jest automatyczne generowanie szkodliwych zapytań do modelu bez ingerencji człowieka czy dostępu do wag modelu docelowego. Jest również bardziej wydajny pod względem zapytań niż GCG, czego skutkiem jest szybsze znalezienie przełamania zabezpieczeń LLM. Model atakujący tworzy niebezpieczne zapytanie do modelu docelowego, którego odpowiedź kierowana jest do modelu oceniającego czy udało się ominąć mechanizmy obronne. W razie niepowodzenia w kolejnej

iteracji model atakujący udoskonala zapytanie na podstawie konwersacji z modelem docelowym, uzyskanego wyniku i własnej analizy. Wyniki przedstawione przez autorów algorytmu wskazują na wysoką wydajność ataku. Technika PAIR osiągnęła skuteczność ataku na poziomie około 50% dla modeli zamkniętych GPT-3.5-Turbo-1106 i GPT-4-0125-preview oraz 73% dla modelu Gemini-Pro. Dla modeli otwartych Vicuna-13Bv1.5 oraz Llama-2-7B-chat otrzymano odpowiednio 88% i 4%. Stanowi to wzrost skuteczności ataków w porównaniu do rezultatów otrzymanych dla metody GCG. Dodatkowo, do przeprowadzenia ataku metoda PAIR wymagała stanowczo mniej zapytań i wykazała wyższy poziom przenośności na inne modele.

Inną metodą wykorzystującą w swoim działaniu duże modele językowe jest atak Crescendo [7]. Jest to wieloturuowy atak czarnoskrzynkowy. Wykorzystuje sekwencję pozornie bezpiecznych zapytań, które stopniowo powodują obejście mechanizmów bezpieczeństwa. Do generowania zapytań wykorzystany jest inny LLM. Atak rozpoczyna się nieszkodliwego pytania zbliżonego do tematyki badanego zagrożenia. W kolejnych turach tej metody atakujący odwołuje się do odpowiedzi wygenerowanych przez atakowany model, co zwiększa prawdopodobieństwo wygenerowania tokenów zakazanych w kolejnej odpowiedzi i tym samym złamania reguł bezpieczeństwa. Autorzy w swoich badaniach wykazali wysoką skuteczność ataku korzystając z tej metody. Dla 50 próbek ze zbioru AdvBench otrzymano binarny wskaźnik sukcesu na poziomie 98% w przypadku ataku na model GPT-4 oraz 100% dla modelu Gemini-Pro. Uzyskano tym samym rezultaty lepsze niż przy użyciu metody PAIR, dla której wartość tego samego wskaźnika była na poziomie odpowiednio 76% oraz 80% dla badanych modeli.

Rozdział 3

Metody obrony przed atakami na duże modele językowe

- 3.1 Taksonomia metod obrony**
- 3.2 Metody na poziomie promptu**
- 3.3 Metody na poziomie modelu**
- 3.4 Metody na poziomie wyjścia**

Rozdział 4

Projekt i implementacja środowiska badawczego

4.1 Cel i założenia eksperymentu

4.2 Architektura systemu testowego

4.3 Wybrane modele językowe

4.4 Zbiory danych

4.5 Metryki oceny

Jedną z metryk wykorzystanych w pracy było Attack Success Rate (ASR). Przy pomocy tej metryki mierzy się procent ataków na model, które z powodzeniem ominęły zabezpieczenia [5]. Metrykę tę definiuje się jako stosunek liczby udanych prób ataku do całkowitej liczby przeprowadzonych prób. Celem wprowadzenia mechanizmów obronnych była minimalizacja tej metryki. Może być zapisana wzorem (1).

$$ASR = \frac{\text{liczba udanych prób ataku}}{\text{liczba prób ataku}} \times 100\% \quad (1)$$

W celu oceny wpływu zaimplementowanych metod mitygacji podatności na wydajność systemu mierzono również opóźnienie. Wnioskowanie AI dzieli się na fazę przetwarzania promptu oraz fazę sekwencyjnego generowania tokenów [1]. Z tego względu latencję analizowano przy użyciu dwóch miar. Pierwszą z nich był czas do wygenerowania pierwszego tokenu (ang. Time To First Token (TTFT)), na który wpływ powinny mieć filtry wejściowe czy modele zewnętrzne. Zdefiniowana jest jako opóźnienie między otrzymaniem żądania a pojawieniem się pierwszego wyjściowego tokenu oraz obejmuje ono czas na planowanie i przetwarzanie zapytania [1]. Jest istotna dla aplikacji, które wymagają wysokiej responsywności [1]. Drugą z kolei metryką jest czas pomiędzy kolejnymi tokenami (ang. Time Between Tokens (TBT)), który jest przydatny przy analizie wpływu wprowadzonych

mechanizmów inżynierii aktywacji, a jej wielkość wpływa na odbieraną przez użytkownika prędkość modelu [1]. By uniknąć maskowania przestojów w generowaniu odpowiedzi i w pełni ocenić wydajność, zgodnie z zaleceniami metodologicznymi [1], nie opierano się na uśrednionym czasie, lecz wyniki interpretowano na podstawie rozkładów statystycznych.

4.6 Implementacja wybranych metod obrony

Rozdział 5

Analiza wyników badań

- 5.1 Skuteczność ataków bez zastosowania metod mitygacji**
- 5.2 Skuteczność poszczególnych metod obrony**
- 5.3 Analiza kompromisu między bezpieczeństwem a użytecznością modelu**
- 5.4 Analiza opóźnień wprowadzanych przez obronę**
- 5.5 Dyskusja wyników i ograniczenia badania**

Rozdział 6

Podsumowanie

6.1 Wnioski

6.2 Możliwości dalszych badań

Bibliografia

- [1] Agrawal, A. i in., *On Evaluating Performance of LLM Inference Serving Systems*, 2025.
- [2] Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G. J. i Wong, E., *Jailbreaking Black Box Large Language Models in Twenty Queries*, 2024.
- [3] Chiang, W.-L. i in., *Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality*, 2023. dostęp 27 lipca 2026. URL: <https://lmsys.org/blog/2023-03-30-vicuna/>
- [4] Choi, W. C., Chang, C. I., Ng, S. I. i Choi, I. C., „A Review of “Do Anything Now” Jailbreak Attacks in Large Language Models: Potential Risks, Impacts, and Defense Strategies”, 2025.
- [5] Correia, P. H. B. i in., *A Systematic Literature Review on LLM Defenses Against Prompt Injection and Jailbreaking: Expanding NIST Taxonomy*, 2026.
- [6] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*, 2025. dostęp 12 lipca 2026. URL: <https://genai.owasp.org/llm-top-10/>
- [7] Russinovich, M., Salem, A. i Eldan, R., *Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack*, 2025.
- [8] Sajadieh, S. i in., „The AI Index 2026 Annual Report”, AI Index Steering Committee, Institute for Human-Centered AI, Stanford University, Stanford, CA, kwiecień 2026. dostęp 12 lipca 2026. URL: <https://arxiv.org/abs/2606.15708>
- [9] Touvron, H. i in., *Llama 2: Open Foundation and Fine-Tuned Chat Models*, 2023.
- [10] Vassilev, A., Oprea, A., Fordyce, A., Anderson, H., Davies, X. i Hamin, M., „Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations”, National Institute of Standards i Technology (NIST), NIST Trustworthy and Responsible AI NIST AI 100-2e2025, 2025.
- [11] Wei, A., Haghtalab, N. i Steinhardt, J., *Jailbroken: How Does LLM Safety Training Fail?*, 2023.
- [12] Yong, Z.-X., Menghini, C. i Bach, S. H., *Low-Resource Languages Jailbreak GPT-4*, 2024.
- [13] Yuan, Y. i in., *GPT-4 Is Too Smart To Be Safe: Stealthy Chat with LLMs via Cipher*, 2024.
- [14] Zhang, C. i in., *Guardians and Offenders: A Survey on Harmful Content Generation and Safety Mitigation of LLM*, 2025.
- [15] Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z. i Fredrikson, M., *Universal and Transferable Adversarial Attacks on Aligned Language Models*, 2023.

Wykaz skrótów

AI Artificial Intelligence 9

ASR Attack Success Rate 17

DAN Do Anything Now 13

GCG Greedy Coordinate Gradient 12

GenAI Generative Artificial Intelligence 9

LLM Large Language Model 9

PAIR Prompt Automatic Iterative Refinement 13

TBT Time Between Tokens 17

TTFT Time To First Token 17

Spis rysunków

Spis tabel